

FAG 确定性计算-多 Batch-Dense-GQA 场景

范杰于斌

2026 年 8 月 11 日

1 额外要求

对于 N_1 个 $m \times n$ 的 batch, 它们有多 batch-Dense 的轮次要求, 还要求对于 $1 \leq p \leq n$, 这些 batch 的第 p 列的轮次不能相同。而这样的 batch 有 b 组, 这 b 组 batch 之间是没有约束的。

首先需要解决的一个重要问题是需要多少个核。答案应该是 $k = \min\{k, bN_1m, bn\}$ 。

粗略地看, 如果直接将 batch 叠在一起, 那么可以得到 b 个 $N_1m \times n$ 的大 batch, 然后调用多 batch 的 dense, 就可以得到一个方案, 但是分析一下它的最大轮次, 先假设所有的核都会被用上, 于是总的轮次为 $N_1m \lceil \frac{bn}{k} \rceil$ 。而非确定性计算的最大轮次是 $\lceil \frac{bN_1mn}{k} \rceil$, 自然地可以考虑是否存在一个方案, 它的轮次为 $m \lceil \frac{bN_1n}{k} \rceil$ 。首先可以分析直接调用 dense 为什么不好, 假设 $bn = tk + \ell (\ell > 0)$, 那么化简为

$$N_1m \lceil \frac{bn}{k} \rceil - m \lceil \frac{bN_1n}{k} \rceil = m(N_1 - \lceil \frac{N_1\ell}{k} \rceil).$$

这个差别可能会特别大, 当 $N_1 \gg k$ 的时候差不多轮次会多出 N_1m 。造成这个现象的原因是, 迫使不同 batch 的相同 ID 列分给了一个核, 明显负载不够均衡。然后看这个最大轮次与非确定性之间的差异:

$$m \lceil \frac{bN_1n}{k} \rceil - \lceil \frac{bN_1mn}{k} \rceil \leq m(\lceil \frac{bN_1n}{k} \rceil - \frac{bN_1n}{k}) < m$$

所以可以得到 $m \lceil \frac{bN_1n}{k} \rceil$ 是所有轮次为 m 的倍数中最接近非确定性最优轮次的。

看例子, $(m, n) = (4, 3), k = 2, b = 3$ 。如果按照上面的叠加拼接的思路然后直接调用多 batch 的 Dense, 那么轮次为 24。但显然有如下方案: 这个例子的轮次是 20, 如果把每个 batch 的每一列看成一个

1	13	5
2	14	6
3	15	7
4	16	8

5	17	9
6	18	10
7	19	11
8	20	12

9	1	13
10	2	14
11	3	15
12	4	16

图 1: 例子

单位, 那它很像 TND 的处理方式。此时我们的方案的最大轮次应该修正为 $m \cdot \max\{\lceil \frac{bN_1n}{k} \rceil, \lceil \frac{n}{m} \rceil, N_1\}$ 。

2 具体思路

2.1 思路一

采用 TND 的思路，先计算 $R = \max\{\lceil \frac{bN_1n}{k} \rceil, \lceil \frac{n}{m} \rceil, N_1\}$ ，最大轮次是 $R_{max} = mR$ 。对于给定的 j, r ，计算全局 ID 为 $ID = (j-1)R + \lceil \frac{r}{m} \rceil$ ，局部的 id 为 $id = r \bmod m(\text{or } m)$ 。如果 $ID > bN_1n$ ，那么直接返回 null。

这里的思路是先把一个 batch 填满再填下一个 batch。所以先调整 ID，因为拼好的 batch 的大小看作是 $num = N_1 \times n$ ，先计算一个 $b_{id} = \lceil \frac{ID}{num} \rceil$ ，然后再让 $ID = ID \bmod num(\text{or } num)$ 。此时就回到了每一个被拼好的 batch。

看什么时候会出现轮次的冲突，计算 $t_1 = \frac{R}{\gcd(N_1, R)}, t_2 = \frac{N_1}{\gcd(N_1, R)}$ ，其中 t_1 是最小的整数使得 $t_1 \cdot N_1$ 是 R 的倍数，即有 $t_1 N_1 = t_2 R$ ，由于我们将一列看成了一个单位，所以即使每 t_1 列会出现轮次重复，但是列中的每个单位自身可以有 m 个轮换保证轮次的不冲突，于是将每 $t_1 m$ 列看成一组，再去做 TND 那样的轮换就可以。判断 $t_1 m$ 是否小于 n ，如果大于等于 n ，那么说明不需要进一步调整。否则需要微调，微调的方法与 TND 类似。

看如何求解坐标：

计算坐标 $w = ID \bmod N_1(\text{or } N_1), y = \lceil \frac{ID}{N_1} \rceil$ 。

若 $t_1 m < n$ ，记录 $n_1 = n \bmod t_1 m(\text{or } t_1 m)$ 。判断 $y > n - n_1$ ，表示不用微调。否则计算 $\delta = \lceil \frac{y}{t_1 m} \rceil$ ，令 $ID = ID + \delta$ ，如果 $ID > \delta \cdot t_2 m R$ ，再令 $ID = ID - t_2 m R$ 。此时再计算 w, y 即可。

并计算在每一列中的偏移量，先算 $y_1 = y \bmod t_1 m(\text{or } t_1 m)$ 。然后 $offset = \lceil \frac{y_1}{t_1} \rceil$ 。最后计算横坐标为 $x = id + offset - 1$ ，判断如果 $x > m$ ，那么让 $x = x - m$ 。最后的 batch 的 id 是 $w + (b_{id} - 1)N_1$ 。

此外，这样做其实是很大范围做轮换，不是很好，其实可以在每 t_1 列做一个轮次的循环。那么步骤为先计算偏移量，然后判断若 $ID > (\delta - 1) \cdot t_2 m R + offset \times t_2 R$ ，再令 $ID = ID - t_2 R$ ，就可以实现。而且这样的轮次调整偏移量没有改变，不需要重新计算。

这样的方案对内存实际上不是特别友好，所以需要调整列分核的策略，就是下面的思路。

2.2 思路二

结合 TND 和多 batch 的思路，先把所有 batch 的第一列铺满，然后再看第二列。那么在计算 ID 后，令 $N = bN_1$ ，表示一个长列的任务块数目，此时 $b_{id} = \lceil \frac{ID \bmod N(\text{or } N)}{N_1} \rceil, y = \lceil \frac{ID}{N} \rceil$ 。

这个时候列冲突的条件就略有不同，因为分配会先把长为 N 的列分配满，所以需要把上面思路中计算轮次冲突中的 N_1 换成 N ，对于在组内的 batchID, w 的计算不变， y 的计算仍要改成 $y = \lceil \frac{ID}{N} \rceil$ 。

3 伪代码

思路二的伪代码

Algorithm 1: dense_batch_GQA($k, m, n, b, N_1, \text{core_id} = j, \text{round_id} = r$)

Require: k : 核总数, m, n : 矩阵维度, b : batch 组数, N_1 : 组内 batch 数目, j : 核心 ID, r : 轮次

Ensure: $w, (x, y)$: 计算得到的位置

```

1:  $k = \min\{k, bN_1m, bn\}$ 
2:  $R = \max\{\lceil \frac{bN_1n}{k} \rceil, \lceil \frac{n}{m} \rceil, N_1\}$ 
3: if  $j > k$  or  $r > Rm$  then
4:   return null
5: end if
6:  $ID = (j - 1)R + \lceil \frac{r}{m} \rceil$ 
7:  $local\_id = r \bmod m$ 
8: if  $local\_id = 0$  then
9:    $local\_id = m$ 
10: end if
11: if  $ID > N_1nb$  then
12:   return null
13: end if
14:  $N = b \times N_1$ 
15:  $b\_id = ID \bmod N$ 
16: if  $b\_id = 0$  then
17:    $b\_id = N$ 
18: end if
19:  $b\_id = \lceil \frac{b\_id}{N_1} \rceil$ 
20:  $y = \lceil \frac{ID}{N} \rceil$ 
21:  $w = ID \bmod N_1$ 
22: if  $w = 0$  then
23:    $w = N_1$ 
24: end if
25:  $g = \gcd(N, R)$ 
26:  $t_1 = \lfloor \frac{R}{g} \rfloor$ 
27:  $t_2 = \lfloor \frac{N}{g} \rfloor$ 
28:  $t_{1\_new} = t_1 \times m$ 
29:  $y_1 = y \bmod t_{1\_new}$ 
30: if  $y_1 = 0$  then
31:    $y_1 = t_{1\_new}$ 
32: end if

```

```

33:  $offset = \lceil \frac{y_1}{t_1} \rceil$ 
34: if  $t_{1\_new} < n$  then
35:    $n_1 = n \bmod t_{1\_new}$ 
36:   if  $n_1 = 0$  then
37:      $n_1 = t_{1\_new}$ 
38:   end if
39:   if  $y \leq n - n_1$  then
40:      $\delta = \lceil \frac{y}{t_{1\_new}} \rceil$ 
41:      $ID = ID + \delta$ 
42:     if  $ID > (\delta - 1) \times t_2 \times m \times R + offset \times t_2 \times R$  then
43:        $ID = ID - t_2 \times R$ 
44:     end if
45:      $w = ID \bmod N_1$ 
46:     if  $w = 0$  then
47:        $w = N_1$ 
48:     end if
49:      $y = \lceil \frac{ID}{N} \rceil$ 
50:   end if
51: end if
52:  $x = local\_id + offset - 1$ 
53: if  $x > m$  then
54:    $x = x - m$ 
55: end if
56: return  $(w + (b\_id - 1) \times N_1, x, y)$ 

```
